

**(15 pts) Questão 1:** A checagem de integridade num sistema pode ser feita de diversas formas. Uma delas é realizar a soma de todos os valores da memória e salvar numa posição. Suponha que o programa e os dados estão gravados na região de memória que vai de 0x100 até 0x2FE. A posição 0x2FF guarda a soma dos valores das memórias de 0x100 até 0x2FE. Crie um **programa** que some todos os valores dos endereços de memória, de 0x100 até 0x2FE e compare o resultado com o valor armazenado em 0x2FF. Se os valores não baterem faça o programa entrar num loop infinito.

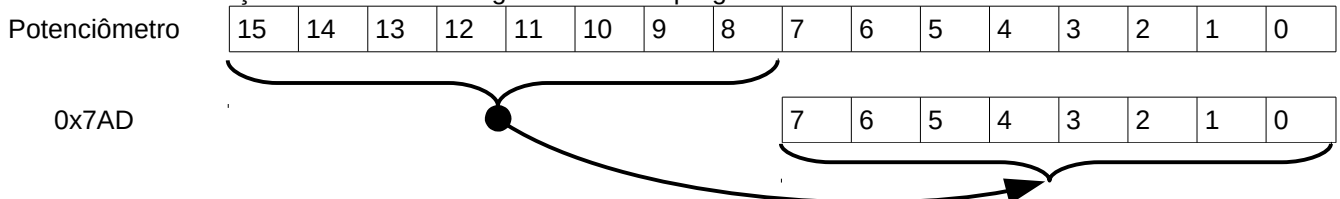
```
Void main (void)
{
    int i, soma;
    char *ptr;
    soma = 0;
    for(i=0x100; i<0x2FF; i++){
        ptr=i;
        soma+= *ptr;
    }
    ptr = 0x2FF;
    if (soma != *ptr){
        for(;;);
    }
}
```

**(15 pts) Questão 2:** O registro CONFIG4L, realiza a configuração de diversos periféricos. Faça sua **configuração** de modo que a programação ICSP com apenas uma fonte esteja ativa, de modo que possa ser possível programar o dispositivo sem necessidade de fonte externa. Além disso habilite a opção de debug e o uso do conjunto de instrução estendida. Apresente o resultado em decimal, hexadecimal e binário. Lembre-se que os registros especiais só podem ser acessados através das diretivas code \_\_\_\_ at \_\_\_\_ = \_\_\_\_\_. Por exemplo:

```
code char at 0x300006 REGISTRO = 0x44;
//em binário 0b010xx1xx
//em decimal 68
```

**(20 pts) Questão 3:** Num dado sistema, o valor de tensão fornecido pelo potenciômetro é lido numa variável inteira de 16 bits não sinalizada. Com base neste valor será realizado o controle de brilho do LCD. Este controle é feito através de uma variável inteira não sinalizada de 8 bits na posição de memória 0x7AD. O brilho do LCD só pode ser configurado com valores entre 0 e 100.

Faça um **programa** que leia o valor do potenciômetro no endereço 0x325, e armazene apenas os oito bits mais significativos no endereço 0x7AD conforme figura abaixo. O programa deve ser cíclico.



```
Void main (void)
{
    unsigned char valor;
    unsigned int *pot;
    unsigned char *lcd;
    pot = 0x325;
    lcd = 0x7AD;
    for(;;){
        valor = (*pot)>>8;
        if (valor >= 100){
            valor = 100;
        }
        *lcd = valor;
    }
}

#define POT (*(unsigned char*)(0x325))
```

REGISTER 25-6: CONFIG4L: CONFIGURATION REGISTER 4 LOW (BYTE ADDRESS 300006h)

| R/P-1 | R/P-0 | R/P-0                | U-0 | U-0 | R/P-1 | U-0 | R/P-1  |
|-------|-------|----------------------|-----|-----|-------|-----|--------|
| DEBUG | XINST | ICPRT <sup>(1)</sup> | —   | —   | LVP   | —   | STVREN |
| bit 7 |       |                      |     |     |       |     | bit 0  |

**Legend:**

R = Readable bit

P = Programmable bit

U = Unimplemented bit, read as '0'

-n = Value when device is unprogrammed

u = Unchanged from programmed state

- bit 7 **DEBUG:** Background Debugger Enable bit  
1 = Background debugger disabled, RB6 and RB7 configured as general purpose I/O pins  
0 = Background debugger enabled, RB6 and RB7 are dedicated to In-Circuit Debug
- bit 6 **XINST:** Extended Instruction Set Enable bit  
1 = Instruction set extension and Indexed Addressing mode enabled  
0 = Instruction set extension and Indexed Addressing mode disabled (Legacy mode)
- bit 5 **ICPRT:** Dedicated In-Circuit Debug/Programming Port (ICPORT) Enable bit<sup>(1)</sup>  
1 = ICPORT enabled  
0 = ICPORT disabled
- bit 4-3 **Unimplemented:** Read as '0'
- bit 2 **LVP:** Single-Supply ICSP™ Enable bit  
1 = Single-Supply ICSP enabled  
0 = Single-Supply ICSP disabled
- bit 1 **Unimplemented:** Read as '0'
- bit 0 **STVREN:** Stack Full/Underflow Reset Enable bit  
1 = Stack full/underflow will cause Reset  
0 = Stack full/underflow will not cause Reset

**Note 1:** Available only in the 44-pin TQFP packages. Always leave this bit clear in all other devices.

```
#define LCD (*(unsigned char*)(0x7AD))
Void main (void)
{
    unsigned char valor;
    for(;;){
        valor = (POT)>>8;
        if (valor >= 100){
            valor = 100;
        }
        LCD = valor;
    }
}
```

**(25 pts) Questão 4:** Num dado sistema, a biblioteca do GPS é composta pelos arquivos "gps.h" e "gps.c" e a biblioteca de controle do carro é "carro.h" e "carro.c". Na biblioteca GPS existem apenas três funções: "void InicializaGPS(void);" que inicializa todas as configurações necessárias, "int LerLatitude(void);" que retorna o valor da latitude e "int LerLongitude(void)". Caso não haja caracter a função retorna o valor 0 (zero). Na biblioteca LCD também temos três funções: "void ControlaVolante(int pos);" que gira o volante para a esquerda, se o número for positivo, e para a direita se for negativo. A função "void ControlaVelocidade(int vel);" recebe um valor indicando em qual velocidade o carro deve andar. Por último a função "void InicializaCarro(void)" responsável por configurar o carro. Supondo que o lugar que o carro será manobrado não possui nenhum obstáculo, construa um **programa**, cíclico, que verifique a posição atual do carro e controle-o para que chegue ao seu destino. A posição destino está definida em duas variáveis "int longDest;" e "int latDest;"

```
#include "gps.h"
#include "carro.h"

//longitude -> esquerda/direita
//latitude-> norte/sul
Void main (void)
{
    int longDest;
    int latDest;
    InicializaCarro();
    InicializaGPS();
    //supondo que o carro começa apontado pra norte
    for(;;){

        // se tiver pra baixo anda pra frente ate chegar.
        while(longDest > LerLongitude()){
            ControlaVelocidade(+1); //anda até chegar na longitude correta
        }

        // se tiver pra cima anda pra tras ate chegar.
        while(longDest < LerLongitude()){
            ControlaVelocidade(+1); //anda até chegar na longitude correta
        }

        //se tiver pra esquerda vira e anda
        if (longDest > LerLongitude()){
            ControlaVolante(-1); //vira pra esquerda
            while(longDest > LerLongitude()){
                ControlaVelocidade(+1); //anda até chegar na longitude correta
            }
        }

        //se tiver pra direita vira e anda
        if(longDest < LerLongitude()){
            ControlaVolante(+1); //vira pra direita
            while(longDest < LerLongitude()){
                ControlaVelocidade(+1); //anda até chegar na longitude correta
            }
        }
    }
}
```

**(25 pts) Questão 5:** Construa uma **biblioteca** chamada "alarmes" que permita ao programador configurar 2 tipos de alarmes: de temperatura e pressão. Os alarmes são ativados pelos bits 3 (temperatura) e 6 (pressão) da porta E. A biblioteca tem quatro funções: uma para configurar o valor limite de temperatura, que recebe um int com o valor limite de temperatura e não retorna nada, uma função para configurar o valor limite de pressão, que recebe um int com o valor limite de pressão e não retorna nada, uma função para inicializar o sistema de alarme e uma função que faz a leitura dos valores de pressão e tensão atuais e aciona as saídas caso algum limite tenha sido ultrapassado. A leitura dos valores de pressão e temperatura pode ser feita através da biblioteca "adc.h" que possui

duas funções: “void InicializaADC(void)” que inicializa o ADC e “int LerValorAD(int canal);” que lê o canal desejado e retorna seu valor. O sensor de temperatura está no canal 3 e o de pressão no canal 6. Lembre-se de fazer os dois arquivos da biblioteca.

```
//alarmes.h
void ValorTemp(int val);
void ValorPres(int val);
void InicializaAlarme(void);
void AtualizaAlarme(void);

//alarmes.c
#include "alarmes.h"
#include "basico.h"
static int alarmeTemp;
static int alarmePres;

void ValorTemp(int val){
    alarmeTemp = val;
}
void ValorPres(int val){
    alarmePress = val;
}
void InicializaAlarme(void){
    InicializaADC();
    BitClr(PORTE,3); //configura como saída o alarme de temperatura
    BitClr(PORTE,6); //configura como saída o alarme de pressão
}
void AtualizaAlarme(void){
    if( LerValorAD(3) > alarmeTemp)
    {
        BitSet(PORTE,3); //liga alarme
    }
    if( LerValorAD(6) > alarmePress)
    {
        BitSet(PORTE,6); //liga alarme
    }
}
```